



F1R3FLY.IO

F1R3GAMES · GAME DESIGN

F1R3Pix

Collective visual intelligence on a F1R3FLY shard

Version 1 · 5 October 2026

Conforms to F1R3Node-Rust master@95be0d4 and the F1R3Games Portal at F1R3Games
v2@2bd746d

SUMMARY

F1R3Pix is a multiplayer game played on a hexagonal grid. Each player controls exactly one hexagon, and the only thing a player can do on the grid is change that hexagon's colour. Players can also message one another and send one another F1R3Cap. Any image larger than one cell therefore has to be negotiated, and the board shows how well a group coordinated.

This document specifies the game against the stack as it stands on 5 October 2026: the F1R3Node-Rust shard, the F1R3Games Portal's wallet, host protocol and environment, and the game environment pattern installed by the Portal service. It corrects the game installed with the Portal, which modelled F1R3Pix as a shared 64×64 canvas any participant could paint anywhere. It adds the two things the Portal lacks for this game: payments between participants, and messages sealed to their recipients. It specifies the gallery record as the whole history of play, since that history can be the outcome of a game, and not only its final frame. Section 16 collects ten decisions, each with a recommendation.

CONTENTS

1	Purpose, sources and conformance	3
1.1	What the game is for	3
1.2	Sources reviewed	3
1.3	Order of authority	3
1.4	Conventions	4
2	What the review found	4
3	The game	5
3.1	Rules	5
3.2	The board	5
3.3	Seating	6
3.4	Colour	7
3.5	Phases	7
4	Architecture	7
4.1	Authority	7
5	The F1R3Pix environment	8
5.1	Configuration	8
5.2	State	8
5.3	Moves	9
5.4	Reads	10
5.5	Order and time	10

6.2	Sealing	12
6.3	What sealing does not protect	12
7	Payments	13
7.1	The payments domain	13
7.2	The amount	13
7.3	The flow	14
8	Host protocol additions	14
9	The client	15
9.1	Layout	15
9.2	Behaviour	16
9.3	Styling	16
10	Galleries and the play record	17
10.1	What is recorded	17
10.2	Encoding	17
10.3	Engagement	17
11	Tournaments	18
12	Security and abuse	18
13	Manifest and conformance	19
14	F1R3Gaze	19
15	Development plan	20
16	Decisions	21
16.1	Out of scope for version 1	21

1 PURPOSE, SOURCES AND CONFORMANCE

1.1 What the game is for

F1R3Pix belongs to the F1R3Games suite of collective intelligence games, *E Pluribus Sapiens*. The suite's premise, from the F1R3Games overview deck of June 2025, is that the actions of any one player are severely limited except for communication: anyone can talk to anyone, and everyone can see the global result of the collective effort. Tokens generalise capital. They amplify or attenuate what a player can achieve, and their first use is in negotiation. The deck's own example is F1R3Pix under its earlier name, PixelFly: to enlist a player, or a block of players, to turn their cells a particular colour so that an image can form, a player sends them tokens.

F1R3Pix is the visual member of the suite. F1R3Beat applies the same discipline to a rhythm, F1R3Ink to the emotional colours players see in one another, F1R3SideChat to a story, and F1R3Skein to a melody.

1.2 Sources reviewed

- The original F1R3Pix design session (9 March 2026), in which the game, its three-column interface and the selection model were specified.
- The F1R3Games overview deck (F1R3GamesOverview.pdf, 20 June 2025), which gives the token rationale and the tournament model.
- The F1R3Games repository: main@ed61943 (the hexagonal prototype, F1R3Pix/UI/F1R3Pix.jsx and its shard service), the branding and preview branches, and v2@2bd746d, which carries the Portal implementation (Portal/f1r3games-portal).
- The F1R3Games Portal design document and the session that produced it (5 October 2026), including its decisions on the wallet, allowances, sponsorships, governance and instance visibility.
- F1R3Node-Rust master@95be0d4 (4 October 2026): system names, SystemVault, TreeHashMap, rho:vault:address and rho:block:data.
- Embers@28e50d4: the pattern in which an environment that moves funds also keeps the authoritative history of what it moved.
- F1R3Gaze and ign1t10n, through the Portal's docs/GAZE-MAPPING.md.
- The brand portal, github.com/F1R3FLY-io/f1r3fly-brand-portal (brand kit v1.2, 20 September 2026), which governs this document's design.

1.3 Order of authority

Where sources disagree, this document follows the order the Portal design set: F1R3Node-Rust master; then Embers; then F1R3Sky; then F1R3Gaze and ign1t10n; then the Portal design and its implementation on v2. The March prototype and the earlier F1R3Games contracts are research. They are the record of what the game is meant to be, and this document keeps their rules and their interface. It does not keep their storage, signing or contract code.

1.4 Conventions

must, must not, should and may are used as in RFC 2119. Requirements are numbered R1, R2, ... and decisions D1, D2, A *participant* is an address in an instance's participants map in the Portal environment. A *seated* participant is one who has been given a cell.

2 WHAT THE REVIEW FOUND

FINDING 1 The installed game environment models a different game

The Portal's templates/games/f1r3pix.rho defines place(instance, x, y, colour) on a square 64×64 canvas. Any participant may paint any pixel, and state is kept per row. That is a shared canvas in the style of r/place. F1R3Pix has one hexagon per player, painted only by its owner, and the move names no coordinates at all. The drift entered on the branding branch in March, where the prototype was rewritten to a square grid with a single global map. The Portal followed that branch. The hexagonal design survives only on main.

FINDING 2 There is no way for a game to pay

The game environments read the caller's identity and never touch a vault, and the wallet refuses game templates that bind deployer authority. Both rules are right. The vault transfer template, though, may be signed only from the Portal shell, and the host protocol (web/src/core/host.ts) offers a game hello, deploy, read, publishPlay, linkPlays, engage and invite. It has no payment method. Sending F1R3Cap to the players selected on the wheel is one of the game's three actions, so F1R3Pix cannot currently be played as designed.

FINDING 3 There is no messaging

Neither the Portal nor the installed environment carries messages between participants. The March shard service wrote messages in plain text to public, string-named inbox channels that any deployer could read or consume.

FINDING 4 There is no record of payments

The Portal sends transfers as bare vault deploys and keeps no history of them. The left column's list of recent transfers has no source.

FINDING 5 The gallery would hold a snapshot, not a play

The installed manifest declares a canvas gallery, and the obvious body for it is the final canvas. The overview deck says that the history of play is recorded and can itself be the outcome. Its examples are an animation of Lucy pulling the football away, and a rose budding and blossoming over the course of a game. A snapshot cannot hold either.

FINDING 6 The prototype left the board's rules open

The March prototype draws 91 cells (radius 5) for 12 players and assigns cells at random in the browser. It does not say what unowned cells are, how seats are allocated, what happens when a player leaves, or whether a token amount sent to several players is split among them or sent to each. Its code splits the amount and silently drops the remainder.

FINDING 7 The prototype's shard code is unsafe to reuse

The March service builds Rholang by string interpolation (an injection risk), keeps one global state per game (so there are no instances), lets the client call `propose`, and uses system names the node has since renamed (`rho:rchain:deployId`, `rho:rchain:revVault`). The Portal already replaces all of this. F1R3Pix adopts the Portal's replacements.

3 THE GAME

3.1 Rules

1. The arena is a hexagonal board of fixed radius, chosen when the instance is created.
2. Each participant may be *seated* at exactly one cell, and each cell seats at most one participant. Cells with no one seated are *void*: part of the picture as background, and unpaintable.
3. The only move on the board is *paint*: a seated participant sets the colour of their own cell. They may do so at any time while the instance is active.
4. Off the board, any participant may send a message to any set of other participants, and may send F1R3Cap to any set of other participants.
5. Everything that happens on the board is recorded. A game's outcome is its history: the final board is one frame of it.

There is no win condition inside a single game. Goals come from outside, from what the players agree to make together or from a tournament (§11).

3.2 The board

Cells are addressed in axial coordinates (q, r) , with the implied third coordinate $s = -q - r$. The board of radius R is

$$B_R = \{(q, r) \in \mathbb{Z}^2 : \max(|q|, |r|, |q + r|) \leq R\}, \quad |B_R| = 3R(R + 1) + 1.$$

Hexagons are drawn pointy-top. A cell's neighbours are its six axial offsets $(+1, 0)$, $(+1, -1)$, $(0, -1)$, $(-1, 0)$, $(-1, +1)$, $(0, +1)$.

radius R	1	2	3	4	5	6	7	8	10	12
cells $ B_R $	7	19	37	61	91	127	169	217	331	469

Every cell also has a *spiral index*, used for compact encodings (§10.2) and for deterministic seating (§3.3). The centre has index 0. Ring k , for $k = 1, \dots, R$, starts at $(-k, +k)$ and walks k steps along each of the six directions in the order listed above. The cells of ring k take indices $3k(k - 1) + 1$ through $3k(k + 1)$.

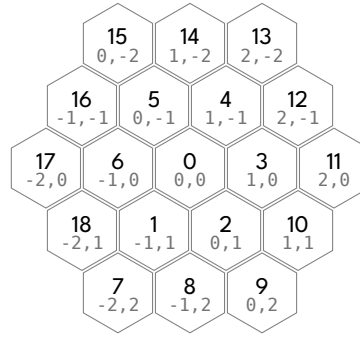


Figure 1: Spiral indices on the board of radius 2. The small figures are the axial coordinates (q, r) .

DECISION D1 How large is the board, and what are empty cells?

Options. (a) The host fixes a capacity N at creation, the board is the smallest B_R with $|B_R| \geq N$, and unseated cells are void. (b) The board grows by a ring when it fills. (c) The board has exactly as many cells as there are players, in an irregular shape.

Recommendation: (a). An image is planned against a fixed frame, and both (b) and (c) move the frame during play. Void cells give the picture a background that no one controls, and a host who wants a full board can set $N = |B_R|$. The default capacity is 61 ($R = 4$). Hosts may choose from 7 to 469 cells ($R \leq 12$) in this version. That bound is set so that a full board read stays a single explore deploy of modest cost, and can be raised once §15 Phase 4 has measured it.

3.3 Seating

A participant is seated by the seat move (§5), which their client makes on first entering an active or lobby instance. Seating is idempotent: a seated participant who calls it again gets their existing cell.

DECISION D2 Who chooses where a player sits?

Options. (a) Random: the environment assigns a cell. (b) Claim: the player chooses a free cell. (c) The host assigns cells.

Recommendation: (a) by default, with (b) as a host option. The deck's premise is that a player's own actions are severely limited, and where you sit is an action with lasting effect on who your neighbours are. Random seating keeps that choice out of the player's hands, so neighbours who have never met must still negotiate. Claim seating suits games among friends and tournaments with set formations. Host assignment adds a step every game would wait on and is left out.

Random seating, precisely. Let $h = \text{blake2b256}(\text{instance} \parallel \text{address})$ read as an unsigned integer. Starting at spiral index $h \bmod |B_R|$, the environment takes the first free cell in increasing index order, wrapping at the end. The result depends only on the instance, the address and the seats already taken, so anyone can check it. A player could grind keys to steer the result. That is acceptable for a game and is noted in §12.

DECISION D3 What happens to a cell when its player leaves?

Options. (a) The cell stays owned and keeps its last colour. (b) The cell becomes free and void. (c) The cell becomes free and keeps its colour for the next occupant.

Recommendation: (a). A player who leaves through the Portal's instances . leave keeps the seat. The client draws the cell dimmed, and rejoining restores control. Freeing cells would let one player hold a sequence of cells by leaving and rejoining, and it would rewrite who painted what in the history. A host who wants to recover seats closes the instance and starts a new one.

3.4 Colour

DECISION D4 Which colours may a cell take?

Options. (a) Any 24-bit sRGB colour. (b) A fixed palette. (c) Any colour by default, with an optional palette set by the host.

Recommendation: (c). On the chain a colour is the string #RRGGBB with upper-case hexadecimal digits. The client offers a palette of 20 swatches, a free picker and the player's recent colours. A host may restrict the instance to a palette of up to 32 colours. That is useful for themed games, for tournaments, and for boards meant to read as pixel art. A cell that has never been painted is *unpainted*, which is distinct from any colour.

3.5 Phases

F1R3Pix uses the Portal's instance status (lobby, active, closed), set only by the host.

DECISION D5 What may players do in each phase?

Recommendation.

	seat	paint	message	pay	read
lobby	yes	no	yes	yes	yes
active	yes	yes	yes	yes	yes
closed	no	no	no	no	yes

The lobby lets players gather, see the frame, sit down, talk and strike deals before anyone paints, so a game can start together, as a tournament needs. Payment stays open in the lobby because that is when deals are made. Closing freezes the board and the record.

4 ARCHITECTURE

F1R3Pix adds no new kind of component to the Portal. It is one game environment on the shard, one game client served from its own origin and framed by the Portal shell, and two additions to the Portal (a payments domain in its environment and two host protocol methods).

4.1 Authority

The design keeps the Portal's separation of authority intact.

- The game client never sees a key. It reaches identity, the shard and payment only through the host protocol.
- The F1R3Pix environment reads the caller's address, which it obtains from `rho:deploy:data` through `rho:vault:address`, and the block height (from `rho:block:data`), and nothing else. It never touches a vault.
- Funds move only through the Portal environment's `payments.send`, signed only from the Portal shell and only after a prompt that names it as a payment.
- Message plaintext is sealed in the game client and opened by the wallet. The chain holds only

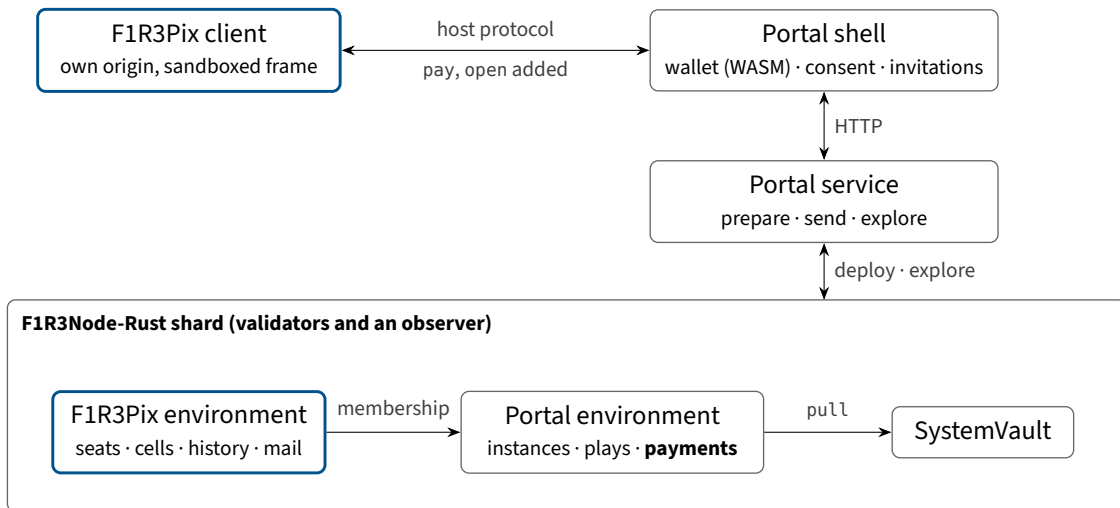


Figure 2: Components. Outlined in dark sky: what this design adds or replaces.

ciphertext.

5 THE F1R3PIX ENVIRONMENT

The environment replaces the installed `f1r3pix.rho` body inside the shared game `prelude.rho`. It keeps the prelude's machinery: installation under the game's registry key with `rho:registry:insertSigned:secp256k1`, version-checked upgrade, `who`, `member`, `instanceOf` and the `TreeHashMap` store. Installing it raises the environment version, so the URI is unchanged and every call template's hash changes. The manifest is then re-registered by the F1R3FLY.io Cooperative (§13).

5.1 Configuration

The instance's configuration is the `config` map the host passes to the Portal's `instances.create`. The environment reads it from the Portal's instance record instead of keeping its own copy, so there is one source of truth. It is fixed at creation.

```

{ "capacity": 61,           // 7 ..= 469; the board is the smallest B_R that seats it
  "seating": "random",      // "random" | "claim" (D2)
  "palette": Nil,          // Nil, or a list of up to 32 "#RRGGBB" (D4)
  "messageLimit": 2048 }   // bytes of sealed envelope per message (sec. 6)

```

R1 The environment must refuse every move on an instance whose configuration is missing or out of range, answering (`false`, `"bad config"`). It must not substitute defaults. Defaults are applied by the client that creates the instance, where the host can see them.

5.2 State

All keys are scoped by instance, so instances never share state.

key	value	written by
("seat", i, a)	{"cell": [q, r], "pk": <public key>, "at": ts, "h": height}	seat, by <i>a</i>
("cell", i, q, r)	{"owner": a, "colour": c Nil, "at": ts, "h": height, "n": count}	seat, paint
("hist", i, q, r)	list of [h, ts, c], oldest first	paint
("out", i, a)	list of envelopes, §6	say, by <i>a</i>

R2 After seating, every key ("cell", i, q, r), ("hist", i, q, r) and ("out", i, a) has exactly one writer: the seated owner, or the sender. Two players therefore never contend for the same key when they paint or send. TreeHashMap spreads keys over a trie keyed by keccak-256 to depth 3, so distinct keys rarely share a leaf either. This is the per-cell successor to the installed environment's per-row storage. It is stronger, because ownership rules out two writers on the same key altogether.

The environment deliberately keeps no instance-wide counter or log, since every writer would contend for it. Global order is recovered when reading (§5.5).

5.3 Moves

Moves are play templates. Within the allowance granted at launch they are signed without a prompt. Each answers (true, value) or (false, reason).

seat(instance, pk, want). Seats the caller. pk is the caller's uncompressed secp256k1 public key, used to seal messages to them. want is Nil under random seating, or [q, r] under claim seating.

- Refuses unless the caller is a participant and the status is not closed.
- Refuses unless rho:vault:address's "fromPublicKey" applied to pk yields the caller's address, so no one can register a key that is not their own.
- If the caller is already seated, answers their cell.
- Under random seating, assigns the cell given by D2. Under claim seating, assigns want if it is on the board and free, and otherwise refuses with "taken".
- Refuses with "full" when every cell is seated.
- Writes the seat and the cell's owner, unpainted. Answers [q, r].

paint(instance, colour). Sets the colour of the caller's own cell. The move names no cell, because the only cell a player can paint is their own.

- Refuses unless the caller is a participant, the status is active, and the caller is seated.
- Refuses unless colour is seven characters: # followed by six of 0123456789ABCDEF. If the instance has a palette, also refuses unless colour is in it.
- Reads the block height *h* and the deploy timestamp *ts*. Writes the cell with *n* incremented, and appends [h, ts, colour] to the cell's history.
- Answers {"cell": [q, r], "colour": colour, "h": h}.

say(instance, to, envelope). Appends a sealed message to the caller's outbox.

- Refuses unless the caller is a participant and the status is not closed.
- Refuses unless to is a non-empty list of at most 64 distinct seated participants, none of them the caller.
- Refuses unless envelope is a byte array no longer than the configured messageLimit.
- Appends {"seq": n, "to": to, "env": envelope, "at": ts, "h": h}, where *n* is the outbox's length before the append. Answers *n*.

5.4 Reads

Reads are explore deploys served by the observer. They cost the reader nothing and need no signature.

template	answers
f1r3pix.board(instance)	{"radius", "capacity", "palette", "status", "cells": [...]}: every seated cell with owner, colour, at, h and n; void cells are absent
f1r3pix.seats(instance)	map from address to {"cell", "pk"}, for highlighting and for sealing
f1r3pix.log(instance, from, to)	every paint with $from \leq h < to$, as [h, ts, owner, q, r, colour], in the order of §5.5
f1r3pix.mail(instance, address, cursor)	every envelope addressed to address after cursor, a map from sender to the last seq seen
f1r3pix.outbox(instance, address, from)	the address's own sent envelopes from seq onwards

Every read result also carries the block hash it reflects, as the Portal's reads do.

5.5 Order and time

A paint is stamped with the height of the block that includes it and the deploy timestamp the painter signed. History is ordered by height, then timestamp, then owner address, then the order of paints within a cell. Height is authoritative: the shard sets it. The timestamp orders paints within a block. It is chosen by the deployer, within the node's acceptance window, so a player can nudge their own paint's position within a block but not move it to another block. For an outcome measured in frames that is immaterial, and the playback renderer (§10) groups by block.

5.6 Contention

Seating is the only move where two players can write the same key: two concurrent random seats can hash to the same free cell, and two claims can name it. The environment reads and writes the cell in one step, so concurrent seats of one cell conflict on that key and are settled by the shard's ordinary conflict handling. The client must treat a seat that was not included, or that answers "taken", as retryable.

5.7 A sketch of paint

```
contract env(@"paint", @instance, @colour, ret) = {
  new whoCh, rCh, sCh, hCh, ack1, ack2 in {
    who!(*whoCh) |
    for (@by, @ts <- whoCh) {
```

```
// participant? status "active"? config valid? colour valid?
ready!(instance, by, "active", colour, *rCh) |
for (@ok, @why <- rCh) {
  if (not ok) { ret!((false, why)) }
  else {
    TreeHashMap!("get", state, ("seat", instance, by), *sCh) |
    for (@seat <- sCh) {
      match seat {
        Nil => { ret!((false, "take a seat first")) }
        _ => { match seat.get("cell") { [q, r] => {
          height!(*hCh) |
          for (@h <- hCh) {
            update!("cell", instance, q, r), colour, ts, h, *ack1) |
            append!("hist", instance, q, r), [h, ts, colour], *ack2) |
            for (_ <- ack1 & _ <- ack2) {
              ret!((true, {"cell": [q, r], "colour": colour, "h": h}))
            }
          }
        } } }
      }
    }
  }
}
}
```

ready, height, update and append are prelude-level helpers; ready performs the participant, status, configuration and colour checks listed above. height is the Portal environment's existing helper over rho:block:data.

6 MESSAGES

6.1 Where messages live

DECISION D6 On the chain or off it, and in the clear or sealed?

Options. (a) On the chain in plain text. (b) On the chain, sealed to the recipients. (c) Relayed off the chain by the Portal service, with only a hash on the chain.

Recommendation: (b). The suite’s premise is that *using F1R3FLY* any player can communicate with any other, so messages belong on the shard, with the same durability and ordering as moves. Plain text (a) would publish every negotiation permanently to anyone who reads the shard. A relay (c) is faster and cheaper, but it puts a server the players must trust between them, and it is not a F1R3FLY channel. Under (b) the sender, the recipient list, the time and the size are public. The content is readable only by the recipients and the sender.

Each message is a deploy, so it costs phlo and arrives with the next block. That is a deliberate trait of the game: talk is cheap but not free, and it moves at the shard's pace. Message moves are play templates and are signed within the allowance without a prompt. The default limit is 2,048 bytes of envelope per

message. That leaves about 1,800 bytes of text for one recipient, since the fixed part of the envelope takes about 110 bytes and each recipient, the sender included, adds a wrap of about 85.

6.2 Sealing

The envelope uses only primitives the Portal wallet already carries (k256, hkdf with SHA-256, aes-gcm), so sealing adds no dependency.

1. The client draws a random 32-byte content key K and an ephemeral secp256k1 key pair (e, E) .
2. The body is

$$C = \text{AES-256-GCM}_K(\text{nonce}_0, \text{plaintext}, \text{aad}), \quad \text{aad} = \text{"f1r3pix/msg/v1"} \parallel \text{instance} \parallel \text{sender}.$$

3. For each recipient j , and for the sender (so the sender can reread their own sent messages), the client computes $Z_j = \text{ECDH}(e, P_j)$ with P_j from `f1r3pix.seats`, derives

$$W_j = \text{HKDF}(Z_j, \text{salt} = E, \text{info} = \text{"f1r3pix/msg/v1"} \parallel \text{instance}),$$

and wraps K as $\text{AES-256-GCM}_{W_j}(\text{nonce}_j, K)$.

4. The envelope is the CBOR map `{v: 1, E, nonce0, C, wraps: [[address, nonce, wrapped K], ...]}`.

Sealing needs only public keys, so the game client does it itself. Opening needs the recipient's private key, so it is done by the wallet through the host protocol's new open method (§8).

R3 The host must pass the wallet the game id and instance of the frame it is hosting, and the wallet must use them in `info` and `aad`. A game frame then cannot use `open` to read envelopes made for another game or another instance. That holds even for the user's own envelopes.

The identity key is a signing key, and `open` also uses it for ECDH. This is the usual ECIES practice, and the `info` string keeps the derived keys separate from every other use. If F1R3FLY later adopts a separate key for encryption, the envelope version changes and `seats` carries that key instead.

6.3 What sealing does not protect

The game client sees the plaintext, because the player types it there. Sealing protects messages from readers of the chain, not from the game client. That is the same trust a player already places in a game they choose to load. Recipient lists are public, so the shape of the negotiation (who talks to whom, how often) is visible to everyone. A player who needs that hidden can use other channels. Hiding recipients is left for a later version (§16.1).

7 PAYMENTS

7.1 The payments domain

Paying is one of the game's three actions, and the Portal has no way for a game to do it (Findings 2 and 4, §2). F1R3Pix needs a payment path that:

- never lets a game sign anything that moves funds;
- lets a game ask for a payment to players it has selected;
- keeps a record of payments within an instance that can be trusted as much as the chain;
- will also serve tournament antes later, and any other game that pays.

It belongs in the Portal environment, which already moves funds for sponsorships through its `pull` helper, and not in the game environment. It follows the Embers pattern: the contract that moves the funds also keeps the history, so the history is as authoritative as the transfers.

`payments.send(instanceId, transfers, memo)`. `transfers` is a list of `[to, amount]`.

- Refuses unless the caller is a participant of the instance and its status is not `closed`.
- Refuses unless there are between 1 and 64 transfers, every `to` is a distinct participant other than the caller, and every amount is a positive integer.
- Refuses unless `memo` is `Nil` or a string of at most 140 characters. The memo is public.
- For each transfer in order, calls `pull` from the caller's vault and records `{"seq", "to", "amount", "ok", "reason", "at", "h", "memo"}` under `("pay", instanceId, caller)`, which only the caller writes.
- Answers the list of per-transfer results.

Transfers are independent. If the vault refuses one (for instance, because the balance ran out), the ones before it stand, and the record says which. The client checks the balance and shows the total before asking, so this case is rare.

`payments.list(instanceId, cursor)`. An explore read. It answers every payment within the instance after `cursor`, merged by height, from the records of each participant.

R4 `payments.send` must be added to the catalogue's `MOVES_FUNDS`. The wallet must sign it only from the Portal origin, always after a prompt, and must not sign it within an allowance.

7.2 The amount

DECISION D7 When several players are selected, what does the amount mean?

Options. (a) Each: the amount is sent to every selected player. (b) Split: the amount is divided among them. (c) Both, chosen by the player.

Recommendation: (c), with each as the default. Most deals in the game are of the form “five each if you all

turn blue”, which is *each*. *Split* suits paying a block for a shared result. The send control shows a toggle and, before anything is signed, the line “10 each to 3 players: 30 F1R3Cap” or “30 split among 3 players: 10 each”. A split that does not divide evenly in the vault’s base unit is refused, and the control suggests the nearest amount that does. The prototype’s silent loss of the remainder is removed.

7.3 The flow

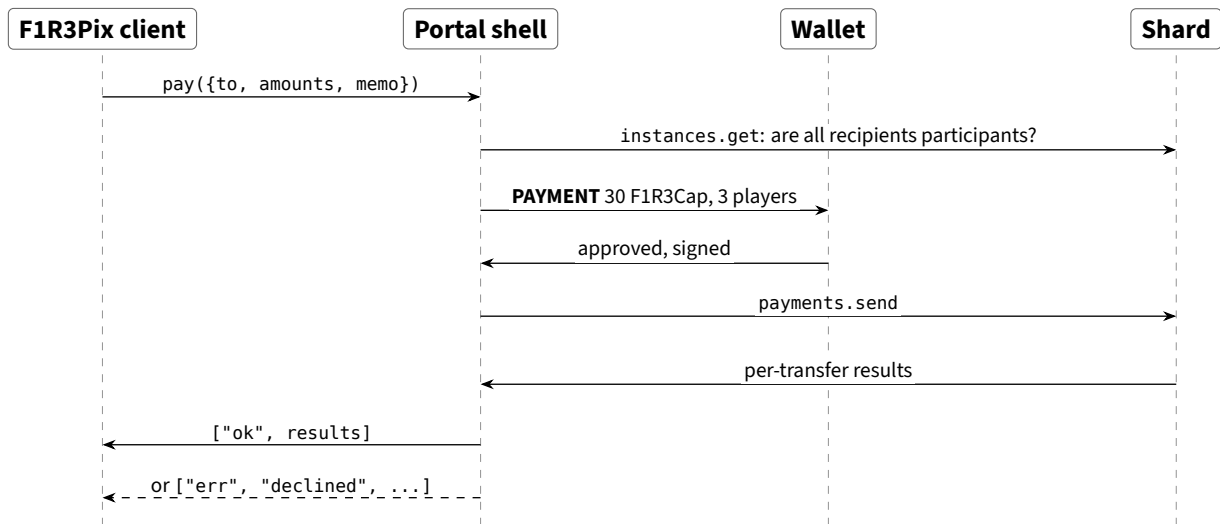


Figure 3: A payment. The game asks; the shell checks the recipients, the person approves, the shell signs and sends.

R5 The shell must refuse a pay request naming anyone who is not a participant of the hosted instance, before showing a prompt. The environment makes the same check on the chain. The shell’s check exists so that a misbehaving game frame cannot even put a prompt for an outside address in front of the player.

8 HOST PROTOCOL ADDITIONS

Two methods are added to `core/host.ts` and to the game SDK. Both follow the protocol’s existing conventions: `["ok", value]` or `["err", code, message]`.

method	behaviour
<code>pay({to, amounts, memo})</code>	Checks that every recipient is a participant of the hosted instance (R5), shows the payment prompt, signs <code>payments.send</code> on approval and sends it. Errors: not-participant, declined, insufficient, node.
<code>open({envelope})</code>	Asks the wallet to open an envelope addressed to the active address, with the hosted game id and instance bound into the derivation (R3). Answers the plaintext. Errors: not-addressed, corrupt. Never prompts.

`open` never prompts because it reveals nothing the player could not already read, and it reveals it only to the game frame they are playing. The wallet must still rate-limit it, at a default of 50 calls per second, so a broken game cannot pin the WASM thread.

Neither method is specific to F1R3Pix. Any game whose manifest declares the capability may use them. The manifest gains a "capabilities": ["pay", "open"] list. The host refuses methods a game has not declared, and the Portal shows a game's capabilities on its card before launch.

9 THE CLIENT

The client is a web page served from the game's own origin and framed by the Portal. To serve the same purpose as the Portal shell, as a test bed for doing this in F1R3Gaze, it follows the shell's structure: a framework-free TypeScript core holding every operation, and a React layer that only renders. In F1R3Gaze each core operation becomes a f1r3lang process over the attenuated portal capability (§14).

9.1 Layout

The page has three columns. The centre column is the widest and holds the board, because the board is where attention belongs.

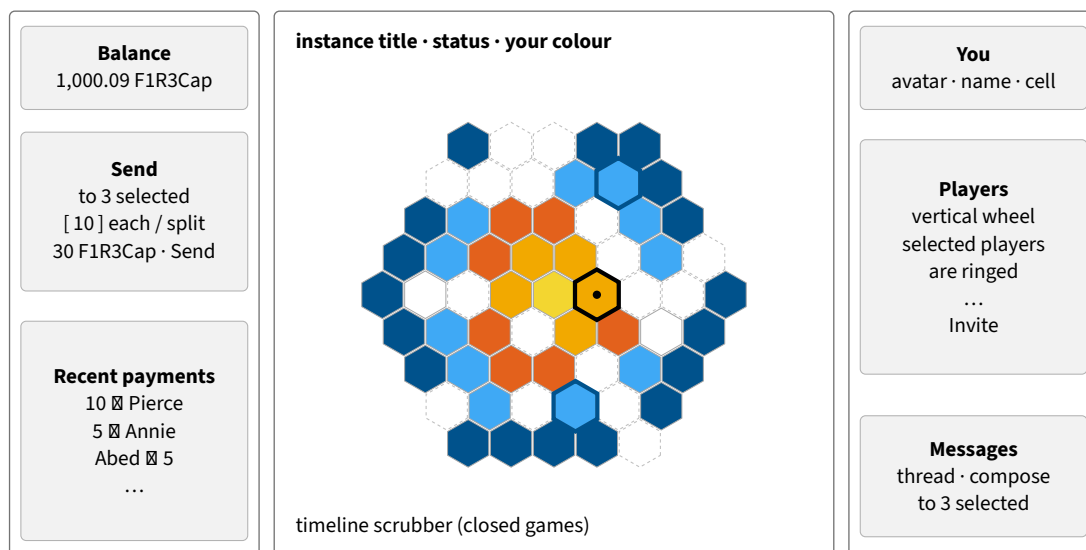


Figure 4: The three columns. In the board, the black-outlined cell with a dot is yours; dark sky outlines mark the selected players' cells; dashed outlines are void cells.

Left column: tokens. The balance, read from the vault. The send control, which always targets the players selected in the right column and shows *each* or *split* with the total (D7). The recent payments within this instance, read from `payments`. List: payments the player sent and received, newest first, abbreviated to a name and an amount.

Centre column: the board. The board fills the column and scales to it. The player's own cell is marked independently of colour, with a black outline and a centre dot. Clicking it opens the colour control: the palette, a free picker, the player's recent colours, and the hex value. Clicking another player's cell toggles that player's selection, so the board is also a selection surface. Hovering shows the owner's name, colour and last change. Selected players' cells are outlined and pulse gently. Void cells are drawn as dashed outlines. Cells of players who have left are dimmed.

Right column: players. The player's own avatar, name and cell at the top. Below it, a vertical wheel of the other players' avatars, drawn from Portal profiles and shrunk to fit: the avatar at the wheel's focus is full size, and avatars shrink with distance from it. Each avatar carries a ring in the player's current cell colour, which ties the wheel to the board. Players may be selected in any number. Selection drives three things at once: highlighting on the board, the recipients of a message, and the recipients of a payment. The wheel's header holds *Invite*, which opens the Portal's shared contact dialogue (the manifest keeps `contactsDialogue: true`). Below the wheel is the message thread, showing messages to and from the selected players, with a compose box addressed to them.

DECISION D8 In what order does the wheel list players?

Options. (a) By distance on the board from the player's own cell. (b) By name. (c) By most recent message or payment.

Recommendation: (a) by default, with (b) and (c) one tap away. Most coordination is with neighbours, and ordering by distance puts the six adjacent players at the wheel's focus.

9.2 Behaviour

- **Freshness.** The client reads board, mail and payments .list once per new block, as reported by the Portal service, and falls back to polling at the configured block interval. Every read carries its block hash, and the client never shows state older than what is on screen.
- **Pending paints.** A paint shows at once, with a pending mark, and settles when a block includes it. While a paint is in flight, further colour choices replace a single queued paint rather than adding more. Dragging through the picker therefore costs one deploy, not dozens.
- **Allowance.** At launch the Portal grants an allowance for `f1r3pix.seat`, paint and say with a budget and an expiry. When it runs out, the next move prompts to extend it. Payments are never inside it.
- **Accessibility.** No information is carried by colour alone. The player's own cell, selection, void and departed cells are all marked by outline, dot or dash. Every cell's hex value is available on hover and to screen readers. The board can be navigated by keyboard in spiral order.
- **Small screens.** Below 900 pixels wide, the three columns become three tabs, with the board as the default tab. The selection persists across tabs.

9.3 Styling

The client's chrome follows the F1R3FLY brand system: a black surface, League Spartan and Source Sans 3, and Yellow and Sky as accents. The board itself carries the players' colours and nothing else. The F1R3Games brand review of 15 June 2026 found the candidate game marks too polished for their audience, and game-level identity is still open. The client therefore uses the F1R3FLY.io system until a F1R3Pix mark is settled.

10 GALLERIES AND THE PLAY RECORD

10.1 What is recorded

DECISION D9 What does a F1R3Pix play hold, and who publishes it?

Recommendation. Keep the registered gallery kind canvas. A canvas play is the board's history over a span of blocks: by default the whole game, or a shorter *moment* that a participant cuts from it, in the way a F1R3Skein performer snips a tune from a performance. Any participant may publish, as the Portal already allows. When the host closes an instance, the client offers to publish the whole game, which takes one prompt. Moments link to the whole game with `linkPlays`, in both directions.

part	content
header	title; span: [from, to] in block heights; blockHash of the state it reflects; radius; players (count); paints (count); preview, the final frame of the span (below)
body	the encoded history of the span (§10.2), stored on the chain as the Portal decided for large bodies, pending the F1R3Drive work

A play is *verifiable*. Anyone can rebuild its body from `f1r3pix.log` at the recorded block and compare hashes, so a published game cannot claim moves that were never made. The preview renderer (`preview/canvas.html`) draws the header's frame. The full view plays the body back with a scrubber, frame by block, so a rose that blossoms over the game, or Lucy pulling away the football, reads as it was played.

10.2 Encoding

The body is a byte array:

```
"F1PX"  u8 version = 1  u8 radius  u32 from  u32 to
u8 P  P × 3 bytes      palette of colours used in the span, in first-use order
varint S  S × (varint Δh, u16 cell, u8 colour)  paints in history order;
                                                Δh from the previous paint's height; cell is the spiral index;
                                                colour indexes the palette, 255 = unpainted
varint O  O × (u16 cell, 20 bytes address)      owners of the cells that appear
```

A span that uses more than 254 colours spills into a second palette page (a palette byte of 254 followed by a u16 index). The header's preview uses the same palette scheme for a single frame: one byte per cell in spiral order. That is at most 469 bytes plus the palette, small enough to list.

10.3 Engagement

Views and likes of plays go through the Portal's engage, and they order the gallery by engagement as well as by recency. F1R3Pix does not breed plays, as F1R3Beat and F1R3Skein do. Its use of engagement is ranking, and later tournament voting.

11 TOURNAMENTS

The overview deck describes tournaments: teams play one or more games, each player pays an ante into a pool, and at the end all players vote gold, silver and bronze across the outcomes, which may be whole histories rather than final frames. Tournaments are outside this version, but the design leaves room for them in four ways:

- The lobby phase lets a game start together (D5).
- The play record is the whole history, and it is verifiable (D9).
- An ante is a payment into an escrow, and the Portal's sponsorship escrow and `payFromEscrow` already do the payout half. A tournament is then a Portal-level object over several instances, not a F1R3Pix feature.
- Engagement already records per-play reactions. A vote is an engagement kind with one vote per participant per tournament.

12 SECURITY AND ABUSE

threat	answer
Painting another player's cell	Impossible: paint names no cell and writes only the caller's seat.
Injection through colours, names or messages	Every template is rendered from typed values by <code>firefly-client</code> . Colours are checked character by character. Messages are opaque bytes.
A game frame moving funds	Games cannot sign payments. <code>send</code> . <code>pay</code> is checked against participants (R5) and always prompts (R4).
A game frame reading other games' mail	<code>open</code> binds the hosted game and instance (R3).
Registering someone else's key for sealing	<code>seat</code> checks the key against the caller's address with <code>fromPublicKey</code> .
Message spam	Every message costs its sender phlo. Recipients can mute a sender, which is client-side. Hosts can use private instances and invitations.
Many keys, many cells (Sybil)	Phlo, capacity, private instances and invitations. Random seating can be steered by grinding keys. For tournaments, claim seating with set formations, or invitation-only instances, removes the incentive.
Harassment in sealed messages	No one but the recipients can read them, so moderation has to start from a recipient. A recipient can disclose a message together with its content key, which proves the content against the envelope on the chain. A <i>report</i> flow built on that is future work.
Metadata	Who messages whom and when is public (§6).

13 MANIFEST AND CONFORMANCE

```
{ "id": "f1r3pix", "name": "F1R3Pix",
  "tagline": "One hexagon each. Make something together.",
  "entry": "<entry-base>/f1r3pix/", "platforms": ["web"],
  "env": "rho:id:...", // unchanged URI; environment version raised
  "templates": [ "f1r3pix.seat", "f1r3pix.paint", "f1r3pix.say", // play
                  "f1r3pix.board", "f1r3pix.seats", "f1r3pix.log",
                  "f1r3pix.mail", "f1r3pix.outbox" ], // explore
  "capabilities": ["pay", "open"],
  "galleries": [ { "kind": "canvas", "renderer": "<entry-base>/f1r3pix/preview/canvas.html"
                  } ],
  "contactsDialogue": true, "readerTier": false, "minProtocol": 2 }
```

The templates `f1r3pix.place` and `f1r3pix.state` are retired. The host protocol version rises to 2, for `pay`, `open` and `capabilities`. A Portal at protocol 1 does not launch F1R3Pix.

System names are those of F1R3Node-Rust master:

- `rho:registry:lookup` and `rho:registry:insertSigned:secp256k1`;
- `rho:deploy:data`, `rho:block:data` and `rho:system:deployId`;
- `rho:vault:address`, for its `"fromDeployerId"` and `"fromPublicKey"` methods;
- `rho:lang:treeHashMap` and `rho:crypto:blake2b256Hash`;
- `rho:vault:system`, in the Portal environment's pull only.

Conformance tests extend the Portal's existing tests:

- every new template and the environment parse with the node's own `rho:lang-rs` parser at the pinned revision (`rho:lint`);
- every play template passes the wallet's rule that game templates bind no deployer authority, and `payments.send` is in `MOVES_FUNDS` and is refused from a game origin;
- the spiral index, random seating and the body encoding round-trip against shared test vectors in Rust and TypeScript;
- the envelope seals in TypeScript and opens in the Rust wallet, and refuses to open under another game id or instance;
- end to end against the mock node: `seat`, `paint`, `say`, `open`, `pay`, and publish a play whose body rebuilds from `log`.

14 F1R3GAZE

In F1R3Gaze a game is a `f1r3lang` page holding an attenuated portal capability instead of a frame speaking `postMessage`. F1R3Pix needs two more methods on that capability than the Portal's

GAZE-MAPPING.md lists:

- pay, attenuated to the instance's participants. This is the pay capability GAZE-MAPPING.md already proposes as one answer to fund-moving installed contracts. F1R3Pix makes it concrete, and the Portal environment's payments.send is the first method declared to go through it.
- open, bound to the game and instance. That needs a wallet operation F1R3Gaze lacks today: ECDH with the identity key under a caller-fixed info string, which never yields the key itself.

GAZE-MAPPING.md should gain both rows when Phase 2 lands.

15 DEVELOPMENT PLAN

phase	work	done when
1 · Environment	New f1r3pix.rho body and prelude helpers; templates; manifest with capabilities; spiral and seating vectors.	Parses; template rules pass; mock-node tests of seat, paint, say and reads.
2 · Portal	payments domain and catalogue entry; wallet open; host pay, open and capability checks; payment prompt; protocol 2.	Payments refused from game origin; open scoped; end to end against the mock node.
3 · Client	TypeScript core and React views; board, wheel, send, thread; pending paints; accessibility; tabs on small screens.	Plays a full game against the mock node with three simulated players.
4 · Live shard	Install on a running shard; measure phlo for each move and read; set the allowance defaults and the capacity bound from the measurements.	One live game of at least seven players, start to close.
5 · Gallery	Encoding; publish at close and moments; preview and play-back renderers; engagement.	A closed game publishes, rebuilds from log, and plays back.
Later	Tournaments; hidden recipients; reports; the F1R3Gaze version.	

Phases 1 and 2 can proceed together. Phase 3 needs the templates from Phase 1 and the host methods from Phase 2, but can begin against stubs.

16 DECISIONS

	question	recommendation
D1	Board size and empty cells	Fixed at creation from a capacity (default 61, up to 469); unseated cells are void.
D2	Seating	Random and verifiable by default; claim seating as a host option.
D3	A player who leaves	Keeps the cell, frozen and dimmed; rejoining restores control.
D4	Colours	Any #RRGGBB; optional host palette of up to 32.
D5	Phases	Lobby: seat, talk, pay. Active: also paint. Closed: read only.
D6	Messages	On the chain, sealed to the recipients and the sender; metadata public.
D7	Amount to several players	<i>Each</i> by default, <i>split</i> on request; uneven splits refused.
D8	Wheel order	Nearest on the board first; by name or by recent contact on request.
D9	The play record	Whole-game history, or a moment cut from it; any participant publishes; host prompted at close.
D10	Where payments live	In the Portal environment as payments . send, with its own record; the game never signs it.

D10 is not set out in a box above because it follows from the Portal's existing rules: games never touch funds, and the contract that moves funds keeps the record of them.

16.1 Out of scope for version 1

Tournaments and antes; hidden recipient lists; a report flow for messages; separate encryption keys; boards larger than 469 cells; animated goals set by the game itself; non-human players; the F1R3Gaze version.